



Database Management Systems

Views and Users

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A view in MySQL is a virtual table.
- It doesn't store actual data by itself but instead stores a query that pulls data from one or more tables (or even other views)
- Primarily used to hide complex queries and simplify into a convenient table like storage
- Since this is stored in memory, we can reuse these views and avoid writing SQL queries multiple times.
- Generally used when we want a table for specific use case or want to assign roles (Example given in further slides)

- To create a view, we can select fields from one or more tables present in the database.
- A view can either have specific rows based on a certain condition or all the rows of a table.
- We define a view in SQL by using the CREATE VIEW command.

CREATE VIEW view1 AS <query expression>;

where <query expression> is any legal query expression.

The view name is represented by view1.

- We can create a view from one table or using joins with multiple tables as well
- If you create a multi join view then only **read** operations are allowed

NOTE: A view is **updatable** (CRUD operations) only if these criterias are met:

- The view is created on a single table only
- Does not contain aggregate functions like sum, max, count etc
- Does not contain distinct, group by, having or union
- Does not have joins or subqueires (query within query like select for example)
- Query is not recursive in nature

Table Schema

Field	Type	Null	Key	Default	Extra
Author_ID	varchar(50)	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Age	int	YES		NULL	
Email	varchar(50)	YES	UNI	NULL	
Bio	text	YES		NULL	
Password	varchar(50)	NO		NULL	

Assume that there is a table called **Authors** with the following Schema and values. We see how views work using this table alone (crud operations)

Table Values

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	35	alice.johnson@example.com	Expert in AI and ML	password123
A002	Bob Smith	40	bob.smith@example.com	Data Scientist	password456
A003	Carol White	28	carol.white@example.com	Researcher in NLP	password789
A004	David Brown	50	david.brown@example.com	Professor in Quantum Computing	password321
A005	Eve Davis	32	eve.davis@example.com	Cloud Computing Specialist	password654
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A007	Grace Lee	30	grace.lee@example.com	Cryptography Enthusiast	password147
A008	Henry Wilson	27	henry.wilson@example.com	Cybersecurity Analyst	password258
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369

Query

```
create view experienced_authors as  
select * from authors  
where age >= 30 and Bio like '%Expert%';
```

```
select * from experienced_authors;
```

Let's create a view to only check experienced authors who are experts (Age ≥ 30 and have the word Expert in their bio)

Result

```
13 • select * from experienced_authors;  
14
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 						
	Author_ID	Name	Age	Email	Bio	Password
▶	A001	Alice Johnson	35	alice.johnson@example.com	Expert in AI and ML	password123
	A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
	A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369

Query

```
-- Inserting a new experienced author
insert into experienced_authors (Author_ID, Name, Age, Email, Bio, Password)
values ("A010", "John Wick", 52, "john.wick@example.com", "Expert in Pencils", "password567");

-- Updating Alice's Age to 40 from 35
update experienced_authors set Age=40 where Author_ID='A001';
```

Performing insert and update operations on the view (Alice age change and john wick added)

Result

```
22 -- Updated Table with changes
23 • select * from experienced_authors;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 						
Author_ID	Name	Age	Email	Bio	Password	
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123	
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987	
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369	
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567	

Query

```
46  -- Deleting Ivy Scott (A009) from the view
47  • delete from experienced_authors where Author_ID="A009" and Name="Ivy Scott";
48  • select * from experienced_authors;
```

Result Grid						
Filter Rows:						
Exports: Wrap Cell Contents:						
Author_ID	Name	Age	Email	Bio	Password	
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123	
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987	
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567	

Deleting Ivy Scott from the view

Authors Table

```
50  -- All the crud operations are reflected on the original table as well
51  • select * from authors;
```

Result Grid						
Filter Rows:						
Edit: Export/Import: Wrap Cell Content:						
Author_ID	Name	Age	Email	Bio	Password	
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123	
A002	Bob Smith	40	bob.smith@example.com	Data Scientist	password456	
A003	Carol White	28	carol.white@example.com	Researcher in NLP	password789	
A004	David Brown	50	david.brown@example.com	Professor in Quantum Computing	password321	
A005	Eve Davis	32	eve.davis@example.com	Cloud Computing Specialist	password654	
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987	
A007	Grace Lee	30	grace.lee@example.com	Cryptography Enthusiast	password147	
A008	Henry Wilson	27	henry.wilson@example.com	Cybersecurity Analyst	password258	
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567	

NOTE: Please note all the changes made to the view is also reflected on the original tables as well
Alice's age has been changed to 40, John wick was added and Ivy Scott was removed from authors table

Example : Create a view that lists each employee's full name, their department name, and the total number of projects they are working on. Then, using the view, write a query to retrieve the names of employees who are working on at least 3 projects.

Query to create a view:

```
CREATE VIEW EmpProjSummary AS
```

```
SELECT E.Fname, E.Lname, D.Dname, COUNT(W.Pno) AS ProjectCount
```

```
FROM EMPLOYEE E JOIN DEPARTMENT D ON E.Dno = D.Dnumber
```

```
JOIN WORKS_ON W ON E.Ssn = W.Essn GROUP BY E.Ssn, E.Fname, E.Lname, D.Dname;
```

View created:

```
mysql> select * from EmpProjSummary;
```

Fname	Lname	Dname	ProjectCount
Jennifer	Wallace	Administration	2
Ahmed	Jabbar	Administration	2
Alicia	Zelaya	Administration	2
James	Borg	Headquarters	1
John	Smith	Research	2
Franklin	Wong	Research	4
Joyce	English	Research	2
Ramesh	Narayan	Research	1

Query the view created:

```
SELECT Fname, Lname, Dname, ProjectCount
```

```
FROM EmpProjSummary
```

```
WHERE ProjectCount >= 3;
```

Fname	Lname	Dname	ProjectCount
Franklin	Wong	Research	4

Example : Create a view that lists employees along with the number of dependents they have. Then, find employees who have more than 2 dependents.

Query to create a view:

```
CREATE VIEW EmpDependentCount AS
```

```
SELECT E.Fname, E.Lname, COUNT(Dp.Dependent_name) AS NumDependents
```

```
FROM EMPLOYEE E
```

```
LEFT JOIN DEPENDENT Dp ON E.Ssn = Dp.Essn
```

```
GROUP BY E.Ssn, E.Fname, E.Lname;
```

View created:

```
mysql> select * from EmpDependentCount;
```

Fname	Lname	NumDependents
John	Smith	3
Franklin	Wong	3
Joyce	English	0
Ramesh	Narayan	0
James	Borg	0
Jennifer	Wallace	1
Ahmed	Jabbar	0
Alicia	Zelaya	0

Query the view created:

```
SELECT * FROM EmpDependentCount
```

```
WHERE NumDependents > 2;
```

Fname	Lname	NumDependents
John	Smith	3
Franklin	Wong	3

- **Complexity:** Views help to reduce complexity. Different views can be created on the same base table for different users.
- **Security:** It increases security by excluding sensitive information from the view.
- **Query Simplicity:** It helps to simplify commands from the user. A view can draw data from several different tables and present it as a single table.
- **Consistency:** A view can present a consistent, unchanged image of the structure of the database. Views can be used to rename columns without affecting the base table.
- **Data Integrity:** If data is accessed and entered through a view, the DBMS can automatically check the data to ensure that it meets the specified integrity constraints.
- **Storage Capacity:** Views take very little space to store the data.
- **Logical Data Independence:** Views can make the application and database tables to a certain extent independent.

- You cannot INSERT if the base table has any not null columns that do not appear in the view.
- You cannot INSERT or UPDATE if any of the columns referenced in the INSERT or UPDATE contains group functions or columns defined by an expression.
- You can't execute INSERT, UPDATE, DELETE statements on a view if with READ ONLY option is enabled.
- You can't create a view using temporary tables.
- You cannot INSERT, UPDATE, or DELETE if the view contains group functions GROUP BY, DISTINCT, or a reference to a pseudo column row_num.
- You can't pass parameters to the SQL server views.
- You can't associate rules and defaults with views.

Database Management Systems

Difference between Views and CTE



Feature	Views	CTEs (Common Table Expressions)
Definition	A virtual table stored in the database, defined by a query.	A temporary result set defined within a query using WITH.
Persistence	Persisted in the database schema until explicitly dropped.	Exists only for the duration of the query execution.
Reusability	Can be reused across multiple queries once created.	Can only be used inside the single query where it is defined.
Scope	Global (available to all users with permissions).	Local (visible only within that query).
Complexity Handling	Good for hiding complex joins, filters, and calculations for repeated use.	Good for breaking down complex queries into readable parts (step-by-step).
Updatability	Some views are updatable (if based on a single table without aggregations).	CTEs are not updatable – they're just temporary results.
Syntax	<code>CREATE VIEW view_name AS <query>; SELECT * FROM view_name;</code>	<code>WITH cte_name AS (<query>) SELECT * FROM cte_name;</code>

In SQL, "role" and "user" are two distinct concepts used for managing access and permissions within a database management system.

User:

- A user is an individual or entity that interacts with the database.
- Users are typically associated with specific individuals or applications that need to perform operations on the database.
- Each user has a unique username and, in some database systems, a password for authentication. Users are granted permissions to perform specific actions (e.g., SELECT, INSERT, UPDATE, DELETE) on database objects (e.g., tables, views).

Role:

- A role is a named group of users or other roles.
- Roles are used to simplify and manage permissions and access control in a database. Instead of granting permissions to individual users, you can assign permissions to roles.
- Users are then added to roles, and they inherit the permissions associated with those roles. This makes it easier to manage and change permissions for multiple users at once.
- Roles can also be nested, meaning you can have roles within roles, creating a hierarchical permission structure.

Here's how roles and users are typically used together:

- You create roles and assign specific permissions to these roles. For example, you might create a "Readers" role with only SELECT permissions and a "Writers" role with INSERT, UPDATE, and DELETE permissions on certain tables.
- You add individual users to the roles they need. For instance, you might add UserA to the "Readers" role and UserB to the "Writers" role.
- Users then inherit the permissions associated with the roles they belong to. So, UserA will have SELECT permissions because they are in the "Readers" role, and UserB will have INSERT, UPDATE, and DELETE permissions because they are in the "Writers" role.

Syntax for Create User:

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

- 'username' : the account name.
- 'host' : where the user can connect from:
 - 'localhost' : only local machine.
 - '%' : any host.
 - Specific IP or hostname possible.
- 'password' : the login password for authentication.

Example: **CREATE USER 'student'@'localhost' IDENTIFIED BY 'stud123';**

To delete the created user:

```
DROP USER 'username'@'host';
```

Example: drop user 'student'@'localhost';;

- MySQL has a feature that provides many control options to the administrators and users on the database.
- We have already learned how to create a new user using the CREATE USER statement in the MySQL.
- Now, we are going to learn about granting privileges to a user account. MySQL provides GRANT statements to give access rights to a user account.
- The SQL standard includes the privileges SELECT, INSERT, UPDATE, and DELETE.

Syntax: **GRANT privileges**
ON database.table
TO 'username'@'host';

Privileges : what the user can do (SELECT, INSERT, UPDATE, etc.).

Database.table : the scope:

- ***.*** : all databases, all tables.
- **mydb.*** : all tables in mydb.
- **mydb.mytable** : specific table.

username@host : user account.

Example: grant all on company to student@localhost;

//student user should be created, the above command grants all privileges to student user

Common Privileges

- **Data privileges:**
 - SELECT, INSERT, UPDATE, DELETE
- **Structure privileges:**
 - CREATE, ALTER, DROP, INDEX
- **Administrative privileges:**
 - GRANT OPTION allows the user to grant privileges to others
 - SUPER is for superuser permissions
 - ALL PRIVILEGES is a shorthand for all available privileges.

Examples: school is the database and students is a table

a) Give read-only access on one table to the user

```
GRANT SELECT ON school.students TO 'alice'@'localhost';
```

a) Give full access on a database to the user

```
GRANT ALL PRIVILEGES ON school.* TO 'bob'@'localhost';
```

a) To check the privileges assigned to a specific user, use the following command

```
SHOW GRANTS FOR 'bob'@'localhost';
```

To revoke an authorization, we use the REVOKE statement.

It takes a form almost identical to that of GRANT.

Syntax:

```
REVOKE privileges  
ON database.table  
FROM 'username'@'localhost';
```

Example:

```
REVOKE select
```

```
ON school.*
```

```
FROM 'alice'@'localhost';
```

Any authorization that can be granted to a user can be granted to a role.

Roles are granted to users just as authorizations are.

Roles can be created in SQL as follows:

```
create role Assistant_employee;
```

Roles can then be granted privileges just as the users :

```
grant select
```

```
on courses
```

```
to Assistant_employee;
```

Roles can be granted to users, as well as to other roles:

```
grant rolename to user_name;
```

Example: `grant Assistant_employee to 'alice'@'localhost';`

Revoking privileges from roles:

```
REVOKE INSERT, UPDATE, DELETE ON database_name FROM role_name;
```

Removing roles:

To delete one or more roles, you use the DROP ROLE statement as follows:

```
DROP ROLE role_name[, role_name, ...];
```

```
DROP ROLE role1, role2;
```

1) Which of the following best describes a "view" in a database?

A) A physical table that stores a duplicate copy of data to speed up queries.

B) A temporary result set defined within a single query using the WITH clause.

C) A virtual table that does not store actual data by itself but instead stores a query that pulls data from one or more tables.

D) A specialized role assigned to users for administrative privileges.

2) Fill in the blank: While a View is persisted in the database schema until it is explicitly dropped, a _____ exists only for the duration of a single query execution.

3) According to the slides, under which condition is a view considered "updatable" (meaning it allows CRUD operations like INSERT or UPDATE)?

A) When it contains aggregate functions like SUM or MAX.

B) When it is created using joins across multiple tables.

C) When it is created on a single table only and does not contain subqueries or the DISTINCT keyword.

D) When it includes a GROUP BY or HAVING clause.

4) Fill in the blank: While the GRANT statement is used to give access rights to a user account, the _____ statement is used to remove an authorization previously given to a user or role.

5) How do views provide increased security in a database environment?

- A) By automatically encrypting all passwords stored in the base tables.
- B) By requiring a secondary username and password every time the view is accessed.
- C) By excluding sensitive information from the view, thereby restricting what data the user can see.
- D) By permanently locking the base tables from any INSERT or UPDATE commands.

Answers:

- 1. C
- 2. CTE (Common table Expression)
- 3. C
- 4. REVOKE
- 5. C

- <https://www.devart.com/dbforge/mysql/studio/create-view-in-mysql.html>
- <https://thanusan25thanu.medium.com/mysql-views-are-not-only-query-able-but-also-updatable-91ac4730bc11>
- <https://dev.mysql.com/doc/refman/8.4/en/views.html>
- <https://www.red-gate.com/simple-talk/databases/mysql/working-with-mysql-views/>
- <https://sqlpad.io/tutorial/ctes-vs-views-in-sql-essential-distinctions-for-tech-interviews>
- <https://learnsql.com/blog/difference-between-sql-cte-and-view>
- <https://minhngocda.medium.com/view-vs-cte-in-sql-difference-and-use-cases-227307d0d672>



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu